

Backtrack recursivo

Motivação

- “Backtrack” significa desfazer decisões anteriores e testar novas opções
- Backtrack recursivo
 - Enumera todas as soluções possíveis (algoritmo força-bruta)
 - Consiste em identificar uma relação de recorrência que fornece a solução ótima (recursão)
- Embora tenha tempo exponencial
 - Podemos tentar acelerar identificando formas de restringir o espaço de busca
 - Em alguns casos conseguimos com isso resolver instâncias reais
 - Em alguns casos conseguimos aplicar programação dinâmica para torná-lo polinomial
 - Ainda assim, para a maioria dos problemas de otimização teremos apenas alg. exponenciais

Estrutura

algoritmo **BacktrackRecursivo**(I)

<pré-cond>: I é uma instância do problema

<pós-cond>: retorna uma solução ótima para I, e seu custo (minimização)

se I é suficiente pequena (caso base)

 retorne a solução e custo deste caso base

senão

 melhorCusto = ∞

 para cada decisão k possível

 Ik = instância que resulta de I após a decisão k

 sol_k, custo_k = **BacktrackRecursivo**(Ik)

 sol, custo = solução e custo p/ I usando decisão k (sol_k e custo_k)

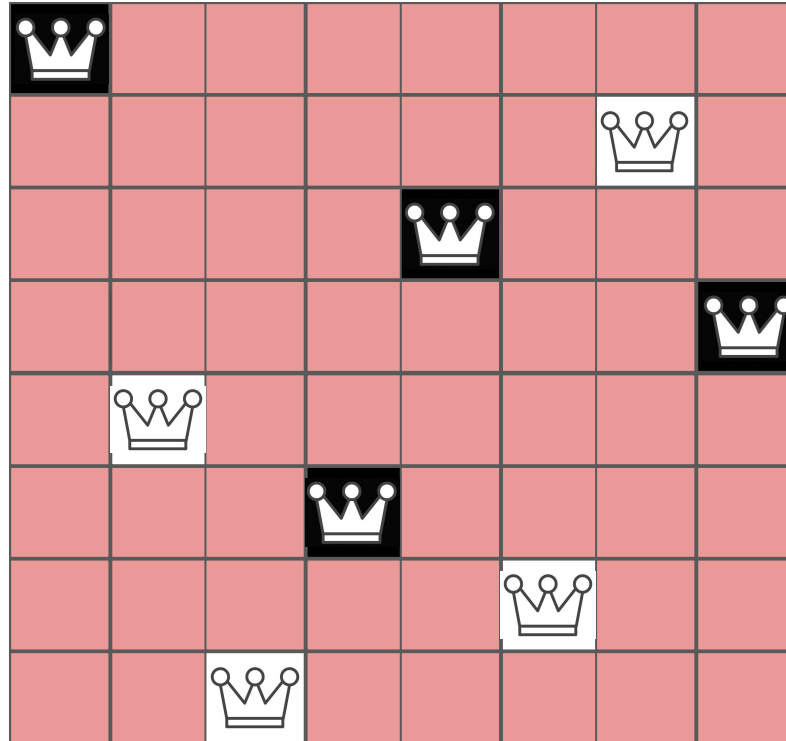
 se custo < melhorCusto

 melhorSol = sol

 melhorCusto = custo

 retorne melhorSol, melhorCusto

Ex.: Problema das rainhas



Passos

1. Especificação

- Instâncias
 - Ex. (rainhas): inteiro n (número de linhas, de colunas e de rainhas)
- Soluções
 - Ex. (rainhas): n posições no tabuleiro das n rainhas
- Medida de sucesso
 - Ex. (rainhas): 0 se existe rainha atacada por outra, ou 1 caso contrário
- Objetivo
 - Ex. (rainhas): maximizar, ou seja, encontrar solução válida

Passos

2. Construindo subinstâncias

- Quais decisões podem reduzir o tamanho da instância?
 - Ex. (rainhas): em qual posição da próxima linha livre colocar a próxima rainha
Esta posição não pode ser atacada pelas rainhas já posicionadas na instância.
- É necessário receber informações adicionais?
 - Ex. (rainhas): sim, precisamos saber as posições das rainhas nas r primeiras linhas que já possuem rainha
- Como construir a subinstância para cada possível decisão?
 - Ex. (rainhas): se a decisão é colocar uma rainha na coluna k da linha $r+1$, acrescentamos esta informação na subinstância

Passos

3. O que receber da chamada recursiva?

- Ex. (rainhas): valor booleano indicando se existe solução válida contendo as rainhas que já foram posicionadas na subinstância (valor), e as posições das rainhas (solução, se houver)

4. Como construir a solução da instância?

- Ex. (rainhas): basta usar a 1a solução de valor 1 retornada na chamada recursiva.
 - Ou conjunto vazio se todas as chamadas recursivas retornam solução de valor 0.

5. Como calcular o custo/valor (medida de sucesso) da solução?

- Ex. (rainhas): 1 se alguma chamada recursiva retorna 1, ou 0 caso contrário.

Passos

6. Como determinar a melhor decisão?

- Ex. (rainhas): qualquer solução com valor 1.

7. Casos base

- Ex. (rainhas): quando todas as n rainhas já foram posicionadas.

Passos

8. Pseudocódigo

algoritmo Rainhas(C,n,r)

<pré-cond>: C é um conjunto de pares (x,y) indicando as posições das rainhas, onde x (de 1 até r) é o número da linha e y (de 1 até n) é o número da coluna. n é o número de linhas, de colunas, e de rainhas que devem ser colocadas. As r primeiras linhas possuem rainhas posicionadas em C.

<pós-cond>: retorna a solução (posições, se existir) e valor da solução (0 ou 1).

se n == r

se nenhuma rainha é atacada em C, retorne (C,1)

senão, retorne ({},0)

senão

para k de 1 até n

C' = C U {(r+1,k)}

(sol, valor) = Rainhas(C', n, r+1)

se valor == 1, retorne (sol, 1)

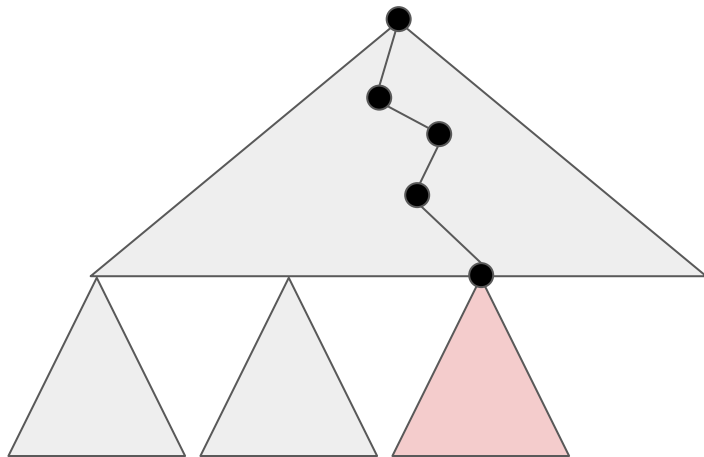
retorne ({}, 0)

Passos

9. Recorrência para o valor da solução ótima

- Ex. (rainhas): $\text{Rainhas}[C, n, r] = \max_{k=1..n} \text{Rainhas}[C \cup \{(r+1, k)\}, n, r+1]$ para $r < n$
 $\text{Rainhas}[C, n, n] = 0$ se rainha é atacada, ou 1 c.c. para $r = n$

Podando a árvore de busca



Podando a árvore de busca

- Soluções inválidas
 - Quando as decisões que já foram feitas impedem a construção de uma solução válida
 - Ex. (rainhas): quando existe linha restante que é toda atacada pelas $r-1$ as rainhas
- Nenhuma solução com valor maior
 - Ex.: maximizar número de elementos, já selecionou 5, faltam 4 para testar
 - Então vai conseguir no máximo valor 9, mas já encontrou solução com valor 10
 - Ex. (rainhas): pode parar a busca quando encontrar solução com n rainhas (valor 1)
- Algoritmo guloso
 - A estratégia guloso já diz qual decisão tomar, descartando os outros ramos da árvore

Podando a árvore de busca

- Existe solução ótima em outros ramos
 - Mostrar que a remoção do ramo não remove todas as soluções ótimas
 - Caso comum é a remoção de soluções simétricas
 - Ex. (rainhas): basta considerar a metade das colunas ao posicionar a rainha da 1ª linha
- Decisões inevitáveis
 - As decisões anteriores podem forçar algumas decisões, então podemos aplicá-las logo
 - Ex. (rainhas): quando sobra apenas uma posição livre em alguma das linhas restantes
- Soluções triviais
 - Quando é possível calcular a solução sem chamadas recursivas
 - Ex.: max num. elementos, elementos restantes são compatíveis entre si e com os já inseridos